

Plan de pruebas de software

Sistema de Gestión Empresarial (Facturación)

Fecha: 13/09/2026

*Hecho por :
Santiago Padierna Suarez*

Tabla de contenido

Historial de versiones.....	4
Información del proyecto.....	4
Aprobaciones.....	4
Resumen ejecutivo	5
Alcance de las pruebas.....	6
Elementos de pruebas.....	6
Nuevas funcionalidades a probar	6
Pruebas de regresión	7
Funcionalidades a no probar	7
Enfoque de pruebas (estrategia)	8
Criterios de aceptación o rechazo.....	9
Criterios de aceptación o rechazo	9
Criterios de suspensión	9
Criterios de reanudación.....	10
Entregables	11
Requerimientos de entornos – Hardware	12
Red y conectividad.....	12
Requerimientos de entornos – Software	12
Herramientas de pruebas requeridas	12
Metodologías y Técnicas de Pruebas Empleadas.....	13
Personal.....	13
Entrenamiento.....	13
Planificación y organización.....	15
Procedimientos para las pruebas.....	15
Matriz de responsabilidades.....	15
Cronograma.....	17
Premisas.....	17
Dependencias y Riesgos.....	18
Referencias.....	19
Glosario.....	19

Historial de versiones

Fecha	Versión	Autor	Organización	Descripción
13/09/2026	V1.0	Santiago Padierna	Institución Universitaria Pascual Bravo	Creación del plan maestro

Información del proyecto

Empresa / Organización	Institución Universitaria Pascual Bravo
Proyecto	Sistema de Gestión Empresarial (Facturación)
Fecha de preparación	13/09/2023
Cliente	Tarrado
Patrocinador principal	Tarrado
Gerente / Líder de proyecto	Santiago Padierna Suarez
Gerente / Líder de pruebas de software	Santiago Padierna Suarez

Aprobaciones

Nombre y Apellido	Cargo	Departamento u organización	Fecha	Firma
Santiago Padierna Suarez	Gerente	Institución Universitaria Pascual Bravo	13/09/2023	

Resumen ejecutivo

Este documento presenta el plan maestro de pruebas de software para el proyecto de desarrollo del Sistema de Gestión Empresarial (Facturación). El propósito fundamental es definir, estructurar y comunicar las estrategias, recursos y cronogramas necesarios para validar que la aplicación cumpla con los requisitos funcionales establecidos en la guía de laboratorio, asegurando que los módulos de Clientes, Productos, Empleados, Facturas e Informes operen de manera correcta y eficiente. Al ser plan maestro, se proporciona una visión de alto nivel de todas las actividades de QA que se llevan a cabo durante el ciclo de vida de desarrollo de la aplicación.

Alcance del plan en relación con el proyecto: Este plan de pruebas está ligado con los objetivos académicos de la asignatura Herramientas de Programación III de la Institución Universitaria Pascual Bravo: implementar un sistema de facturación funcional con persistencia de datos en base de datos relacional (PostgreSQL/Supabase) y una arquitectura de código organizada (MVC).

Alcance del esfuerzo de pruebas: El esfuerzo de testing se centrará en:

- Pruebas funcionales: validación de los CRUD de Clientes, Productos y Empleados, y del flujo completo de creación de facturas (selección de cliente, agregado de productos al detalle, cálculo de totales y cambio de estado de la factura).
- Pruebas de integración: verificación de la comunicación correcta entre la aplicación (Python/Tkinter) y la base de datos (Supabase), incluyendo los procedimientos almacenados de actualización/eliminación por entidad.
- Pruebas de validación de datos: comprobación de campos obligatorios, tipos de dato y restricciones (por ejemplo, no permitir facturas sin productos o cantidades negativas).

Restricciones:

- Recursos tecnológicos: al ser un proyecto de un solo desarrollador, no se cuenta con un ambiente de pruebas (staging) separado del de desarrollo; las pruebas se ejecutan directamente sobre la base de datos de Supabase en uso.
- Tiempo: el ciclo de pruebas debe completarse antes de la fecha límite de entrega de la asignatura, sin margen para retrasos.
- Presupuesto y entorno: la validación será completamente manual y exploratoria, sin herramientas de automatización, dado que el proyecto no cuenta con presupuesto para licencias de software de pruebas.

Alcance de las pruebas

Elementos de pruebas

1. Formulario de Login
2. Dashboard (pantalla principal)
3. Tabla de Clientes (con barra de búsqueda)
4. Formulario de Cliente
5. Tabla de Productos (con barra de búsqueda)
6. Formulario de Producto
7. Tabla de Categorías de Producto (con barra de búsqueda)
8. Formulario de Categoría de Producto
9. Tabla de Facturas (sin barra de búsqueda)
10. Formulario de Factura
11. Tabla de Informes (sin barra de búsqueda)
12. Formulario de Informe
13. Tabla de Empleados (con barra de búsqueda)
14. Formulario de Empleado
15. Tabla de Roles de Empleado (con barra de búsqueda)
16. Formulario de Rol de Empleado
17. Formulario de creación de Administrador
18. Botón de Ayuda / Acerca de

Nuevas funcionalidades a probar

1. **Login**
 - a. Inicio de sesión con usuario y clave válidos.
 - b. Manejo de credenciales inválidas (mensaje de error).
2. **Dashboard**
 - a. Visualización correcta al ingresar tras el login.
 - b. Navegación desde el dashboard hacia los demás módulos.
3. **Clientes**
 - a. Listado de clientes en tabla y búsqueda por nombre/documento.
 - b. Registro de un nuevo cliente con campos obligatorios.
 - c. Edición y eliminación de un cliente existente.
4. **Productos**
 - a. Listado de productos en tabla y búsqueda por nombre/código.
 - b. Registro de producto asociado a una categoría.
 - c. Edición de precio, stock y datos del producto.
5. **Categorías de Producto**
 - a. Listado de categorías en tabla y búsqueda por descripción.
 - b. Creación, edición y eliminación de categorías.
6. **Facturas**
 - a. Listado de facturas en tabla (sin buscador).
 - b. Selección de cliente y empleado para la factura.
 - c. Adición de productos al detalle con cantidad y precio válidos (mayores a cero).
 - d. Cálculo correcto del total (descuento, impuesto).
 - e. Asignación/cambio de estado de la factura.
7. **Informes**
 - a. Listado/consulta de informes en tabla (sin buscador).
 - b. Visualización de datos de facturas (número, fecha, cliente, empleado, estado).

8. **Empleados**
 - a. Listado de empleados en tabla y búsqueda por nombre/documento.
 - b. Registro de empleado con rol asignado.
 - c. Edición de datos y fecha de retiro.
9. **Roles de Empleado**
 - a. Listado de roles en tabla y búsqueda por descripción.
 - b. Creación, edición y eliminación de roles.
10. **Creación de Administrador**
 - a. Registro de un nuevo usuario administrador con acceso al sistema.
11. **Ayuda / Acerca de**
 - a. Verificar que despliegue correctamente la información de la aplicación.
12. **Control de acceso por rol**
 - a. Verificar que al iniciar sesión, el sistema muestre únicamente los módulos/formularios permitidos según el rol del empleado (Gerente, Secretaria, Operario, Administrativo).
 - b. Verificar que un rol con permisos limitados no pueda acceder (ni ver el botón/menú) a los módulos restringidos.
 - c. Verificar que el administrador tenga acceso a todos los módulos, incluyendo creación de administradores y roles.

Pruebas de regresión

1. **Impacto de cambios en Productos sobre Facturas**
 - Verificar que al modificar el precio o eliminar un producto, las facturas ya creadas con ese producto no pierdan su información en el detalle de factura.
2. **Impacto de cambios en Roles sobre Empleados**
 - Comprobar que modificar o eliminar un rol no deje empleados con referencias inválidas.
3. **Impacto de cambios en Clientes sobre Facturas**
 - Validar que editar los datos de un cliente no afecte las facturas históricas asociadas a él.
4. **Consulta de informes tras cambios de estado**
 - Verificar que el módulo de Informes refleje correctamente el estado actualizado de una factura después de haberlo modificado.
5. **Impacto de cambios en Roles sobre el acceso a módulos**
 - Verificar que modificar los permisos/descripción de un rol se refleje correctamente en qué módulos ve un empleado con ese rol, sin afectar el acceso de empleados con otros roles.

Funcionalidades a no probar

Eliminación en cascada de clientes/productos con facturas asociadas

- Razón: no está definido el comportamiento (bloquear o eliminar en cascada) cuando el cliente o producto tiene facturas relacionadas.
- Riesgos asumidos: posible error de integridad referencial no controlado por la interfaz.

Cifrado de contraseñas

- Razón: no se definió requisito de cifrado para el almacenamiento de credenciales.
- Riesgos asumidos: vulnerabilidad de seguridad si la base de datos es expuesta.

Pruebas de carga y concurrencia

- Razón: proyecto académico monousuario, sin requisito de múltiples usuarios simultáneos.
- Riesgos asumidos: ninguno relevante para el alcance actual.

Eliminación en cascada de clientes/productos con facturas asociadas

- Razón: no está definido el comportamiento (bloquear o eliminar en cascada) cuando el cliente o producto tiene facturas relacionadas.
- Riesgos asumidos: posible error de integridad referencial no controlado por la interfaz.

Cifrado de contraseñas

- Razón: no se definió requisito de cifrado para el almacenamiento de credenciales.
- Riesgos asumidos: vulnerabilidad de seguridad si la base de datos es expuesta.

Pruebas de carga y concurrencia

- Razón: proyecto académico monousuario, sin requisito de múltiples usuarios simultáneos.
- Riesgos asumidos: ninguno relevante para el alcance actual.

Enfoque de pruebas (estrategia)

1. **Tipos de pruebas**
 - a. Funcionales (caja negra): validar cada CRUD y flujo contra los criterios de la guía de laboratorio.
 - b. De control de acceso: verificar que cada rol vea solo los módulos permitidos.
 - c. De integridad de datos: validar campos obligatorios y restricciones numéricas (cantidad y precio mayores a cero en el detalle de factura).
 - d. De integración: confirmar que las operaciones desde la app reflejen correctamente los cambios en la base de datos.
2. **Configuraciones a probar:** entorno único (PC del desarrollador conectado a Supabase), sin variación de dispositivos.
3. **Subconjuntos de datos:** reutilización de datos de ejemplo ya cargados, más casos límite (cantidad = 0, precio negativo) y al menos un usuario de prueba por cada rol.
4. **Nivel de regresión:** moderado-alto — cada cambio en Productos, Clientes o Roles exige reprobación los flujos de factura y de control de acceso.

Criterios de aceptación o rechazo

Criterios de aceptación o rechazo

Criterios de aceptación para los módulos operativos

1. 100% funcional el registro, edición y eliminación en cada CRUD (Clientes, Productos, Categorías, Empleados, Roles).
2. Funcionamiento correcto del inicio de sesión, la barra de búsqueda en las tablas correspondientes, y el listado de datos.
3. 0 errores críticos. Se toleran fallas menores (ej. problemas de alineación visual, textos o formato de fecha) siempre que no impidan usar la aplicación y queden anotadas para corregir después.
4. Acceso correcto al dashboard y navegación fluida entre módulos tras iniciar sesión.

Criterios de rechazo para los módulos operativos

1. Un registro creado (cliente, producto, empleado, etc.) no aparece o aparece incompleto en su respectiva tabla.
2. Hay errores en las búsquedas de las tablas o en la edición/eliminación de registros.
3. El sistema se cierra solo o no permite iniciar sesión.
4. Más del 20% de las pruebas realizadas fallan.

Criterios de aceptación para el módulo de Facturación e Informes

1. 100% en la creación de facturas, cálculo correcto del total (incluyendo descuento e impuesto) y asignación del estado de la factura.
2. 100% en la visualización de los datos de la factura (cliente, empleado, estado) en el módulo de Informes.
3. Control de accesos validado: un empleado sin permisos no puede acceder a los módulos restringidos según su rol.

Criterios de rechazo para el módulo de Facturación e Informes

1. Una factura creada no aparece o se muestra incompleta (sin cliente, sin detalle de productos) en la tabla de Facturas o en Informes.
2. El cálculo del total de la factura es incorrecto (descuento, impuesto o suma del detalle).
3. Fallos de seguridad que permiten a un empleado sin permisos acceder a módulos restringidos, crear administradores o modificar roles.

Criterios de suspensión

1. La aplicación se cierra sola (crash) al abrirla, o el módulo de inicio de sesión falla de forma generalizada, impidiendo entrar al sistema.
2. La base de datos (Supabase) o la conexión se caen de forma constante, impidiendo guardar información de clientes, productos, empleados o facturas.
3. Imposibilidad total de agregar productos al detalle de una factura, o fallos que bloqueen por completo el módulo de Facturación.

4. El control de acceso por rol no carga o no permite verificar qué módulos ve cada empleado, imposibilitando probar el flujo completo de permisos.
5. Si en la primera hora o en las primeras ejecuciones falla entre el 20% y el 30% de los casos de prueba intentados de forma consecutiva.

Criterios de reanudación

1. El desarrollo adjunta un informe breve indicando que el fallo bloqueante (ej. el login o la conexión con la base de datos) fue resuelto y verificado.
2. Se ejecuta exitosamente una prueba de humo de 10-15 minutos para verificar que el sistema vuelve a ser estable antes de retomar el plan de pruebas completo.

Entregables

Antes de las pruebas

- **Plan de pruebas:** define el alcance, estrategia, criterios, riesgos y cronograma del proyecto de testing.
- **Matriz de Trazabilidad:** vincula los requisitos de la guía de laboratorio (CRUDs, facturación, control de acceso por rol) con sus respectivos casos de prueba para garantizar cobertura total.
- **Especificación y Diseño de Casos de Prueba:** matriz estructurada con el detalle paso a paso de los casos diseñados (precondiciones, datos de entrada, pasos y resultado esperado).

Durante las pruebas

- **Logs de Errores y Registro de Evidencias:** capturas de pantalla que respaldan la ejecución y el comportamiento del sistema.
- **Reportes de Incidencias / Defectos (Bugs):** fichas de los errores encontrados (pasos para reproducir, severidad, prioridad y estado).

Al finalizar la fase de pruebas

- **Informe Final de Pruebas:** documento con el resumen de la ejecución, métricas alcanzadas (porcentaje de casos exitosos/fallidos) y estado de los defectos.
- **Acta de Aprobación:** documento breve que formaliza la aceptación del sistema para su entrega/sustentación de la asignatura.

Recursos

Requerimientos de entornos – Hardware

- **Base de datos:** se recomienda usar una segunda base de datos en Supabase (o un esquema separado) dedicada a pruebas, distinta de la que contiene los datos reales, para no afectar información en caso de errores durante la ejecución. *(Restricción: por presupuesto, es posible que se use la misma instancia; en ese caso, se debe respaldar la BD antes de probar.)*
- **Equipo de cómputo:** 1 computador con procesador de gama media (Intel i5 o equivalente), 8 GB de RAM en adelante, suficiente para ejecutar Python, Tkinter y la conexión a Supabase sin problemas de rendimiento.

Red y conectividad

- **Conexión a internet:** conexión estable de banda ancha, necesaria para la comunicación con la base de datos en Supabase (servicio en la nube).
- **Acceso de red / VPN:** no aplica — Supabase se accede directamente mediante cadena de conexión, sin necesidad de VPN.

Requerimientos de entornos – Software

Accesos a sistemas y entornos

- **Credenciales de entorno de pruebas:** cuentas de prueba para cada rol definido (Gerente, Secretaria, Operario, Administrativo) para validar el control de acceso por rol.
- **Acceso a la base de datos de pruebas:** permisos de lectura/escritura sobre la BD en Supabase para verificar la persistencia de clientes, productos, empleados y facturas.

Software instalado en el equipo de pruebas

- **Entorno de ejecución:** Python 3.x con las librerías del proyecto (Tkinter, conector a PostgreSQL) instaladas.
- **Editor/IDE:** VSCode, para revisar código y depurar errores durante las pruebas.
- **Herramienta de gestión de pruebas:** no se usará una plataforma dedicada (Jira/TestRail); los casos de prueba y defectos se registrarán en una hoja de cálculo (Excel/Google Sheets) por restricciones de presupuesto.
- **Navegadores web:** no aplica — no es una aplicación web.
- **Sistema operativo:** Windows 10/11 (el usado para el desarrollo).

Herramientas de pruebas requeridas

Gestión de Pruebas y Defectos

- Hoja de cálculo (Excel / Google Sheets): para la matriz de trazabilidad, el diseño de casos de prueba y el registro del ciclo de vida de defectos (severidad, prioridad, estado). Se usa esto en lugar de Jira/TestRail por restricciones de presupuesto.

Pruebas de Base de Datos

- pgAdmin / consola SQL de Supabase: para verificar directamente en la base de datos que las operaciones de la app (CRUDs, procedimientos almacenados) persistan correctamente los datos.

Automatización de Pruebas

- No se utilizará automatización (Selenium/Cypress/Appium) — la aplicación es de escritorio (Tkinter), no web ni móvil, y las pruebas se ejecutarán de forma manual.

Pruebas de Rendimiento y Carga

- No aplica — proyecto académico monousuario, sin requisito de concurrencia.

Monitoreo e Inspección

- Consola/terminal de Python: para revisar mensajes de error o excepciones no controladas durante la ejecución de la app.

Metodologías y Técnicas de Pruebas Empleadas

- **Pruebas de Caja Negra:** evaluación del comportamiento funcional del sistema desde la perspectiva del usuario, sin requerir análisis del código fuente.
- **Partición de Equivalencia y Valores Límite:** aplicada en campos como cantidad y precio en el detalle de factura (validando valores en 0, negativos y positivos).
- **Transición de Estados:** para validar el ciclo de vida de la factura (Pendiente Pago → Pagada → Anulada / Vencida).
- **Pruebas Exploratorias:** sesiones no estructuradas para descubrir fallos inesperados en la navegación entre formularios y tablas.
- **Pruebas de Humo (Smoke Testing):** verificación rápida de estabilidad (login, dashboard, conexión a base de datos) después de cada corrección o cambio en el código.

Personal

Una (1) persona: Santiago Padierna, quien cumple simultáneamente los roles de desarrollador, líder de pruebas y tester funcional. No hay equipo de automatización ni de pruebas no funcionales/API, dado que el proyecto no las requiere (ver Enfoque de pruebas).

Entrenamiento

- **Flujos de negocio del módulo de Facturación:** repaso de las reglas de creación de factura, cálculo de totales (descuento e impuesto), y transición de estados (Pendiente Pago → Pagada → Anulada / Vencida), tomadas de la guía de laboratorio.

- **Módulo de Control de Acceso por Rol:** revisión de qué módulos debe ver cada rol (Gerente, Secretaria, Operario, Administrativo) antes de diseñar los casos de prueba.
- **Criterios de negocio y reglas de validación:** aclaración de reglas no documentadas explícitamente en la guía, como el manejo de duplicados en clientes/productos o el comportamiento ante intentos fallidos de login.

Capacitación en herramientas de pruebas

- **Registro en hoja de cálculo:** definición del formato estándar para casos de prueba (precondición, pasos, resultado esperado) y para el registro de defectos (severidad, prioridad, estado), ya que no se usa Jira/Azure DevOps.
- No se requiere capacitación en Postman ni JMeter, dado que el proyecto no expone API ni requiere pruebas de carga.

Planificación y organización

Procedimientos para las pruebas

La ejecución del plan de pruebas se regirá por una metodología de pruebas funcionales de caja negra, estructurada en las siguientes fases:

- **Fase 1: Análisis y Diseño de Casos de Prueba**
 - Revisión de la guía de laboratorio y de las pantallas ya construidas (login, dashboard, tablas y formularios).
 - Diseño de casos de prueba con estructura: precondition, pasos, datos de entrada y resultado esperado, para cada módulo (Clientes, Productos, Categorías, Empleados, Roles, Facturas, Informes, Control de acceso).
- **Fase 2: Preparación del Entorno y Datos**
 - Verificación de la conexión a la base de datos en Supabase.
 - Carga de datos de prueba (clientes, productos, empleados de ejemplo) y creación de un usuario por cada rol, para validar el control de acceso.
- **Fase 3: Ejecución de Pruebas**
 - Ejecución manual y secuencial de cada caso de prueba sobre la aplicación.
 - Comparación entre el resultado obtenido y el resultado esperado definido en el criterio de aceptación de cada módulo.
- **Fase 4: Reporte y Gestión de Defectos**
 - Registro inmediato de cualquier falla encontrada en la hoja de cálculo designada (pasos para reproducir, severidad, prioridad, estado).
 - Captura de pantalla como evidencia de cada defecto.
- **Fase 5: Retesting y Pruebas de Regresión**
 - Repueba de los defectos una vez corregidos.
 - Ejecución del conjunto de pruebas de regresión definido (impacto de Productos sobre Facturas, de Roles sobre Empleados y sobre el control de acceso, de Clientes sobre Facturas).
- **Fase 6: Cierre de Pruebas**
 - Evaluación de si se cumplieron los criterios de aceptación o se incurrió en un criterio de suspensión.
 - Emisión del informe final documentando el estado de calidad de la aplicación antes de la entrega/sustentación.

Matriz de responsabilidades

Al ser un proyecto de una sola persona, no aplica una matriz RACI multirol. Todas las responsabilidades (elaboración del plan, diseño de casos, preparación del entorno, ejecución, reporte de defectos e informe final) recaen en Santiago Padierna, quien actúa simultáneamente como desarrollador y líder/analista de pruebas.

Tarea / Actividad	Santiago (Desarrollador)	Santiago (Líder/Tester QA)
Elaboración del Plan de Pruebas	C	R / A
Diseño de Casos de Prueba	C	R / A
Preparación del Entorno y Datos de Prueba	R	A
Ejecución de Pruebas Funcionales (CRUDs)	C	R / A
Ejecución de Pruebas de Facturación y Control de Acceso	C	R / A
Reporte y Seguimiento de Defectos	I	R / A
Corrección de Defectos Reportados	R	C
Retest y Pruebas de Regresión	C	R / A
Elaboración del Informe Final y Cierre de QA	I	R / A

Descripción de responsabilidades por rol:

- **Santiago (Desarrollador):** encargado de construir/corregir el sistema; prepara el entorno técnico y corrige los defectos que el rol de QA le reporta.
- **Santiago (Líder/Tester QA):** encargado de diseñar y ejecutar los casos de prueba, reportar defectos, validar las correcciones y emitir el informe final que determina si el sistema cumple los criterios de aceptación.

Cronograma

Fase/Actividad	Descripción	Predecesor a	Duración est.	Hito relacionado
1. Planificación y análisis	Revisión de requisitos y diseño del plan de pruebas	N/A	1 día	Hito 1: Plan de pruebas aprobado
2. Diseño de casos de prueba	Elaboración de casos funcionales por módulo	Fase 1	1 día	Hito 2: Casos de prueba completos
3. Configuración del entorno	Verificación de conexión a BD y carga de datos de prueba	Fase 2	1 día	Hito 3: Entorno estable y validado
4. Ejecución funcional	Ejecución de casos manuales en Clientes, Productos, Categorías, Empleados, Roles	Fase 3	2 días	Hito 4: Cobertura del 100% de casos planificados
5. Ejecución de Facturación y control de acceso	Ejecución de casos sobre Facturas, Informes y permisos por rol	Fase 3	2 días	Hito 5: Módulo crítico validado
6. Retest y regresión	Verificación de correcciones y suite de regresión	Fase 4, 5	1 día	Hito 6: 0 defectos críticos pendientes
7. Cierre de pruebas	Consolidado de métricas e informe final	Fase 6	1 día	Hito 7: Entrega del informe final

Premisas

- **Estabilidad del entorno:** se asume disponibilidad continua de la base de datos en Supabase durante el ciclo de pruebas.
- **Requisitos:** se asume que la guía de laboratorio y los requerimientos definidos por el docente representan la versión final a validar.
- **Disponibilidad de recursos:** se asume que el desarrollador contará con el tiempo definido en el cronograma sin interrupciones externas.
- **Metodología:** el proceso se basa en pruebas de caja negra, sin requerir automatización.

- **Calidad de entrada:** se asume que cada módulo llega a la fase de pruebas habiendo pasado una verificación básica de que abre sin errores (smoke test previo).
- **Datos de prueba:** se contará con un poblado inicial coherente (clientes, productos, empleados de ejemplo) para validar los cálculos y flujos de facturación.

Dependencias y Riesgos

Categoría	Descripción del Riesgo / Dependencia	Probabilidad	Impacto
Dependencias con Desarrollo	Retrasos en la corrección de defectos, ya que la misma persona desarrolla y prueba (no hay paralelismo).	Media	Alto
Requerimientos y Ambigüedad	Vacíos de especificación o reglas no escritas en la guía de laboratorio (ver Análisis de la prueba).	Alta	Alto
Disponibilidad de Recursos	Indisponibilidad o caída del servicio de Supabase durante las pruebas.	Baja	Alto
Restricciones de Tiempo	Ventana de tiempo insuficiente para completar toda la regresión antes de la fecha de entrega académica.	Media	Medio
Premisas y Asunciones	Que los datos de prueba cargados no cubran todos los casos límite necesarios (cantidades, precios, roles).	Baja	Medio

Referencias

- Guía de laboratorio, asignatura Herramientas de Programación III, Institución Universitaria Pascual Bravo.
- Script de migración de base de datos (DBFACTURAS: SQL Server → PostgreSQL/Supabase).

Glosario

- **CRUD**: operaciones básicas de Crear, Leer, Actualizar y Eliminar sobre una entidad (cliente, producto, etc.).
- **RACI**: matriz de asignación de responsabilidades (Responsable, Aprobador, Consultado, Informado).
- **Caso de prueba**: conjunto de condiciones, datos de entrada y pasos definidos para verificar que una funcionalidad se comporte como se espera.
- **Defecto (bug)**: comportamiento del sistema que no coincide con el resultado esperado definido en los criterios de aceptación.
- **Pruebas de Regresión**: ejecución de casos ya aprobados para asegurar que cambios recientes no hayan dañado funciones que ya operaban correctamente.
- **QA (Quality Assurance)**: aseguramiento de la calidad; conjunto de actividades planificadas para garantizar que la aplicación cumpla los estándares definidos.
- **Testing**: proceso metódico de evaluación del software para validar sus funcionalidades y encontrar defectos antes de su entrega.
- **Smoke Test (prueba de humo)**: verificación rápida de que las funciones más críticas (login, conexión a BD) funcionan antes de continuar con pruebas más detalladas.